```
# Train.py:

import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers,models

IMAGE_SIZE = (224,224)
Batch = 32

train_path = "D:\\Internship\\Day54\\dataset\\dataset\\train"

datagen = ImageDataGenerator(rescale=1./255,validation_split=0.2)

train_ds = datagen.flow_from_directory(train_path,
                                       target_size = IMAGE_SIZE,
                                       batch_size = Batch,
                                       class_mode = "categorical",
                                       subset = "training")

val_ds = datagen.flow_from_directory(train_path,
                                     target_size = IMAGE_SIZE,
                                     batch_size = Batch,
                                     class_mode = "categorical",
                                     subset = "validation")

model = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu',input_shape = (224,224,3)),
    layers.MaxPool2D(),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPool2D(),
    layers.Conv2D(128,(3,3),activation = 'relu'),
    layers.MaxPool2D(),
    layers.Flatten(),
    layers.Dense(128,activation='relu'),
    layers.Dense(train_ds.num_classes,activation='softmax')])
```

```
model.compile(optimizer='adam',loss='categorical_crossentropy',
            metrics=['accuracy'])

model.summary()

model.fit(train_ds, epochs=10, validation_data = val_ds)

model.save('dog_breed_model.h5')
print("Model saved successfully")

# Test.py:

import tkinter as tk
from tkinter import filedialog
from PIL import Image,ImageTk
import numpy as np
import tensorflow as tf
import json
from tensorflow.keras.preprocessing import image

model = tf.keras.models.load_model('dog_breed_model.h5')

with open('labels.json','r') as f:
    class_indices =json.load(f)

labels = list(class_indices.keys())

def predict_image(img_path):
    img = image.load_img(img_path,target_size = (224,224))
    img=image.img_to_array(img)
    img = img/255.0
    img = np.expand_dims(img, axis=0)

    result = model.predict(img)
    idx = np.argmax(result)
```

```
    return labels[idx]

def open_image():
    global img_path, panel

    img_path = filedialog.askopenfilename(
        filetypes = [("Image files","*.jpg *.jpeg *.png")])

    img = Image.open(img_path)
    img = img.resize((300,300))
    img_tk = ImageTk.PhotoImage(img)

    panel.config(image = img_tk)
    panel.image = img_tk

    result.config(text="")


def classify():
    if img_path:
        label = predict_image(img_path)
        result.config(text = f"Prediction: {label}")

root = tk.Tk()
root.title("Dog Breed Classifier")
root.geometry('400x500')

panel = tk.Label(root)
panel.pack(pady=10)

btn_load = tk.Button(root,text = "Load Image", command=open_image)
btn_load.pack(pady = 10)

btn_predict = tk.Button(root, text = 'Classify',command=classify)
```

```
btn_predict.pack(pady = 10)


result = tk.Label(root,text="",font=("Arial",20))
result.pack(pady=20)


img_path = ""


root.mainloop()
```

Outputs:

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 64)	0
conv2d_2 (Conv2D)	(None, 52, 52, 128)	73,856
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 128)	0
flatten (Flatten)	(None, 86528)	0
dense (Dense)	(None, 128)	11,075,712
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 11,170,250 (42.61 MB)
Trainable params: 11,170,250 (42.61 MB)
Non-trainable params: 0 (0.00 B)
```

```
Epoch 1/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 35s 1s/step - accuracy: 0.1097 - loss: 2.6001 - val_accuracy: 0.2810 - val_loss: 2.2372
Epoch 2/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 22s 1s/step - accuracy: 0.3968 - loss: 1.8467 - val_accuracy: 0.7124 - val_loss: 1.0303
Epoch 3/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 19s 922ms/step - accuracy: 0.8645 - loss: 0.5218 - val_accuracy: 0.9542 - val_loss: 0.1805
Epoch 4/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 18s 874ms/step - accuracy: 0.9919 - loss: 0.0439 - val_accuracy: 1.0000 - val_loss: 0.0046
Epoch 5/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 18s 883ms/step - accuracy: 1.0000 - loss: 0.0035 - val_accuracy: 1.0000 - val_loss: 6.9916e-04
Epoch 6/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 17s 855ms/step - accuracy: 1.0000 - loss: 4.9998e-04 - val_accuracy: 1.0000 - val_loss: 2.4481e-04
Epoch 7/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 18s 882ms/step - accuracy: 1.0000 - loss: 1.2489e-04 - val_accuracy: 1.0000 - val_loss: 8.4122e-05
Epoch 8/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 18s 880ms/step - accuracy: 1.0000 - loss: 7.3679e-05 - val_accuracy: 1.0000 - val_loss: 6.1836e-05
Epoch 9/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 17s 859ms/step - accuracy: 1.0000 - loss: 6.1378e-05 - val_accuracy: 1.0000 - val_loss: 5.3608e-05
Epoch 10/10
20/20 ━━━━━━━━━━━━━━━━━━━━ 17s 861ms/step - accuracy: 1.0000 - loss: 5.4263e-05 - val_accuracy: 1.0000 - val_loss: 4.8407e-05
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is
he native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model, 'my_model.keras')`.
Model saved successfully
```

Dog Breed Classifier
Load Image
Classify
Prediction: German_Shepherd